Write a Custom JDBC User Store Manager

Introduction

A company often already has a user database, with its own way of hashing passwords. You do not have to convert that database to the WSO2 Identity Server standard. Instead, you write a **custom user store manager**: a Java class that tells WSO2 Identity Server how to read, write and check users in that store.

In this lab you build, deploy and test the WSO2 sample **custom-jdbc-user-store-manager**. It keeps the standard JDBC tables, but hashes and checks passwords with the third-party library **Jasypt** instead of the built-in SHA-256 hashing.

Part	What it does
CustomUserStoreManager	Extends UniqueIDJDBCUserStoreManager (the standard JDBC user store). Overrides two methods: preparePassword (hash a new password with Jasypt) and doAuthenticateWithUserName (check a password at login with Jasypt).
CustomJDBCUserStoreMgtComponent	An OSGi component. When the bundle starts, it registers CustomUserStoreManager as a user store type, so the Console can offer it.
[user_store_mgt] custom_user_stores	deployment.toml setting that allows the new class as a user store type.

Time: about 45 minutes.

CC BY 4.0

Prerequisites

1. WSO2 Identity Server 7.3.0 (`<IS_HOME>` below) and JDK 21 with `JAVA_HOME` set.
2. Apache Maven 3.6 or later, and Git. The machine needs internet access to `https://maven.wso2.org` and Maven Central, because the build downloads the WSO2 libraries.
3. MySQL 8 running on port 3306, and the MySQL Connector/J JAR in `<IS_HOME>/repository/components/lib` (as in Lab 01-02).

Step 1: Get the sample

1. Clone the WSO2 samples repository (or download it as a ZIP from GitHub and extract it). The folder is `<SAMPLE_IS>` below:

   ```
   git clone https://github.com/wso2/samples-is.git
   ```

2. The sample is in `<SAMPLE_IS>/user-mgt/custom-jdbc-user-store-manager`. It has two classes:

   ```
   src/main/java/org/wso2/custom/user/store/CustomUserStoreManager.java
   src/main/java/org/wso2/custom/user/store/internal/
   CustomJDBCUserStoreMgtComponent.java
   ```

Step 2: Read the code

You do not need to change the code. These excerpts show the important parts (imports are left out).

The user store manager class

The class is in the package `org.wso2.custom.user.store` and extends the

CC BY 4.0

standard JDBC user store. The constructor with six arguments is the one WSO2 Identity Server calls when it loads the user store:

```
package org.wso2.custom.user.store;

public class CustomUserStoreManager extends UniqueIDJDBCUserStoreManager {

    private static final Log log =
LogFactory.getLog(CustomUserStoreManager.class);
    private static final StrongPasswordEncryptor passwordEncryptor =
            new StrongPasswordEncryptor();

    public CustomUserStoreManager() {
    }

    public CustomUserStoreManager(RealmConfiguration realmConfig,
            Map<String, Object> properties, ClaimManager claimManager,
            ProfileConfigurationManager profileManager, UserRealm realm,
            Integer tenantId) throws UserStoreException {
        super(realmConfig, properties, claimManager, profileManager,
realm, tenantId);
        log.info("CustomUserStoreManager initialized...");
    }
```

preparePassword is called when a user is created or a password is changed. It returns the value stored in the `UM_USER_PASSWORD` column. Jasypt adds its own random salt inside the hash, so the salt from WSO2 Identity Server is not used:

```
    @Override
    protected String preparePassword(Object password, String saltValue)
            throws UserStoreException {
        if (password != null) {
            String candidatePassword = String.copyValueOf(((Secret)
password).getChars());
            log.info("Generating hash value using jasypt...");
            return passwordEncryptor.encryptPassword(candidatePassword);
        } else {
            log.error("Password cannot be null");
```

CC BY 4.0

```
            throw new UserStoreException("Authentication Failure");
        }
    }
```

doAuthenticateWithUserName is called at login. It reads the user's row with the `SelectUserNameSQL` query of the user store, then lets Jasypt compare the entered password with the stored hash. The main lines are:

```
    @Override
    public AuthenticationResult doAuthenticateWithUserName(String
userName,
            Object credential) throws UserStoreException {
        ...
        String sql =
realmConfig.getUserStoreProperty(JDBCRealmConstants.SELECT_USER_NAME);
        ...
        userID = rs.getString(1);              // UM_USER_ID
        String storedPassword = rs.getString(3); // UM_USER_PASSWORD
        isAuthenticated =
passwordEncryptor.checkPassword(candidatePassword, storedPassword);
        ...
        log.info(userName + " is authenticated? " + isAuthenticated);
        // returns SUCCESS with the user, or FAIL with "Invalid
credentials."
    }
```

The OSGi component

WSO2 Identity Server runs on OSGi. This component starts with the bundle and registers the new user store manager as a service. The `@Reference` makes it wait for the RealmService (user management) to be ready:

```
package org.wso2.custom.user.store.internal;

@Component(name = "custom.jdbc.user.store.mgt.component", immediate =
true)
public class CustomJDBCUserStoreMgtComponent {
```

CC BY 4.0

```
    @Activate
    protected void activate(ComponentContext ctxt) {
        UserStoreManager customUserStoreManager = new
CustomUserStoreManager();

ctxt.getBundleContext().registerService(UserStoreManager.class.getName(),
                customUserStoreManager, null);
        log.info("CustomUserStoreManager bundle activated
successfully..");
    }

    @Reference(name = "RealmService",
            service =
org.wso2.carbon.user.core.service.RealmService.class,
            cardinality = ReferenceCardinality.MANDATORY,
            policy = ReferencePolicy.DYNAMIC,
            unbind = "unsetRealmService")
    protected void setRealmService(RealmService rlmService) {
        realmService = rlmService;
    }
    ...
}
```

Step 3: Build the bundle

1. Go to the sample folder and build it:

   ```
   cd <SAMPLE_IS>/user-mgt/custom-jdbc-user-store-manager
   mvn clean install
   ```

2. When the build finishes with `BUILD SUCCESS`, the bundle is in the `target` folder: `custom-jdbc-user-store-manager-<version>.jar` (for example `custom-jdbc-user-store-manager-4.6.5-SNAPSHOT.jar`). The Jasypt classes are packed inside it, so you do not copy Jasypt separately.

CC BY 4.0

Step 4: Create the database for the custom user store

1. Open the MySQL client as root (`mysql -u root -p`) and run:

```
CREATE DATABASE custom_db;
CREATE USER IF NOT EXISTS 'wso2user'@'localhost' IDENTIFIED BY
'Wso2@123';
GRANT ALL PRIVILEGES ON custom_db.* TO 'wso2user'@'localhost';
FLUSH PRIVILEGES;

USE custom_db;
SOURCE <IS_HOME>/dbscripts/mysql.sql;
```

   Replace `<IS_HOME>` with the full path, using forward slashes also on Windows (for example `C:/wso2/wso2is-7.3.0`). `dbscripts/mysql.sql` is the standard user store script (it creates 60 tables, including `UM_USER`). If `wso2user` already exists from another lab, the `CREATE USER` line does nothing.

Step 5: Deploy the bundle

1. Stop WSO2 Identity Server if it is running.
2. Copy the JAR from Step 3 into `<IS_HOME>/repository/components/dropins`.
3. Open `<IS_HOME>/repository/conf/deployment.toml` and add at the end:

```
[user_store_mgt]
custom_user_stores =
["org.wso2.custom.user.store.CustomUserStoreManager"]
```

   The value is the full class name (package and class) of the user store manager.

CC BY 4.0

If the file already has a `[user_store_mgt]` section, add the line to that section instead.

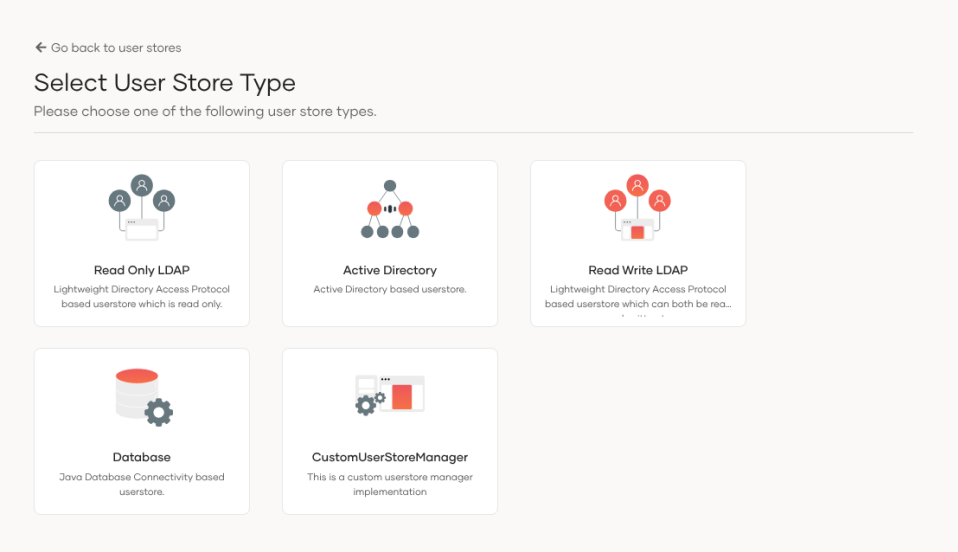
4. Start WSO2 Identity Server: `wso2server.bat` (Windows) or `sh wso2server.sh` (Linux and macOS) from `<IS_HOME>/bin`.
5. Check the startup log. This line shows that the bundle started and the OSGi component registered the user store manager:

```
[2026-11-14 13:34:04,907]  INFO {org.wso2.custom.user.store.internal.CustomJDBCUserStoreMgtComponent} - CustomUserStoreManager bundle activated successfully..
```

CC BY 4.0

Step 6: Create a user store that uses the custom manager

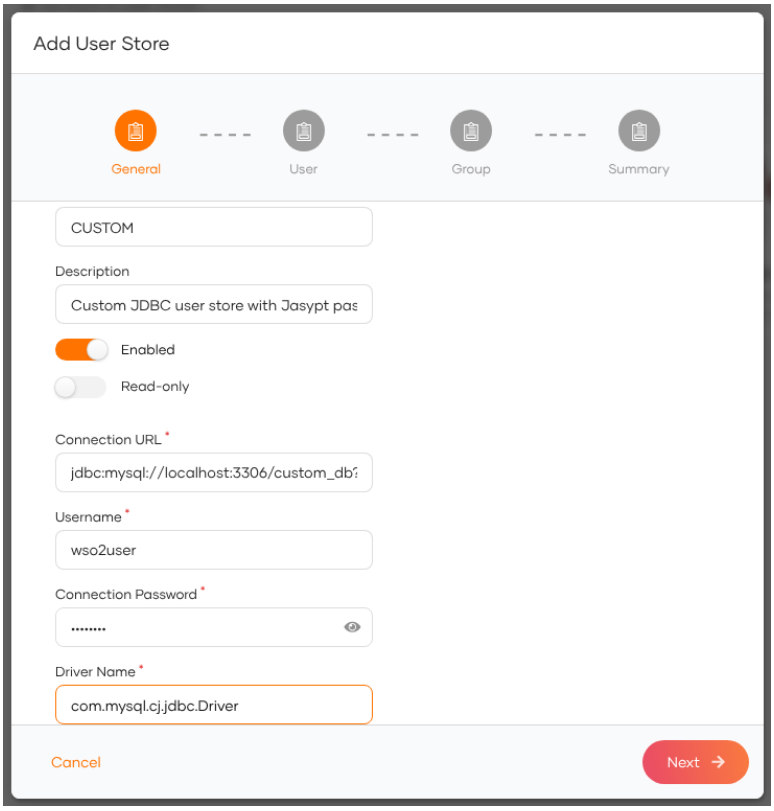
1. Log in to the Console (`https://localhost:9443/console`) as `admin` / `admin`.
2. Go to **User Attributes & Stores > User Stores** and click **+ New User Store**. A new type, **CustomUserStoreManager**, is listed next to the standard types. Select it.



CC BY 4.0

3. On the **General** page, enter:

Field	Value
Name	CUSTOM
Description	Custom JDBC user store with Jasypt password hashing
Connection URL	jdbc:mysql://localhost:3306/custom_db?useSSL=false&allowPublicKeyRetrieval=true
Username	wso2user
Connection Password	Wso2@123
Driver Name	com.mysql.cj.jdbc.Driver



CC BY 4.0

4. Click **Next** on the **User** and **Group** pages (keep the default values), then click **Finish** on the **Summary** page. The Console shows **User store added successfully!**. Refresh the page after a few seconds to see **CUSTOM** in the list.
5. The server log now shows that WSO2 Identity Server created the user store with your class:

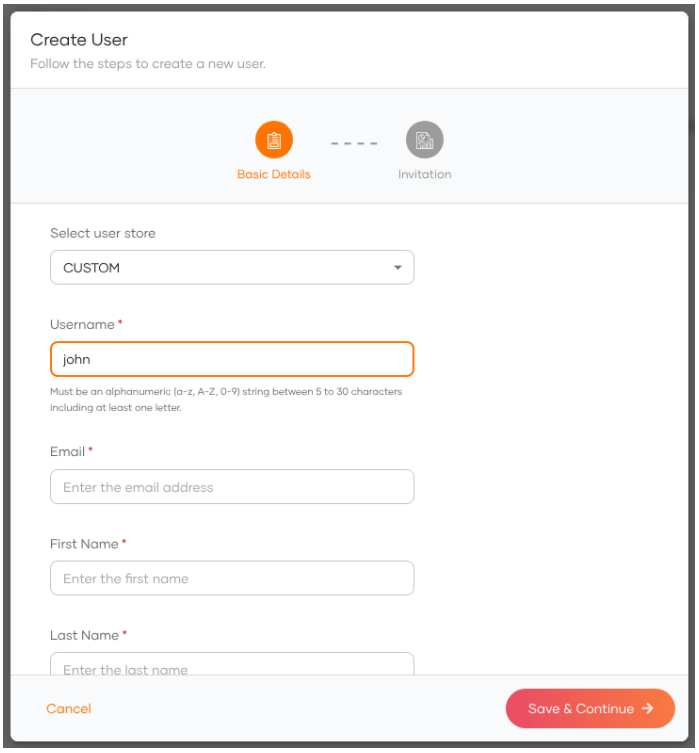
```
[2026-11-14 13:37:59,336]  INFO {org.wso2.custom.user.store.CustomUserStoreManager} - CustomUserStoreManager initialized...
```

CC BY 4.0

Try It

A. Create a user: the password is hashed with Jasypt

1. Go to **User Management > Users**, click **Add User > Single User**.
2. Select **CUSTOM** under **Select user store**. Enter the username `john`, the email `john@example.com`, the first name `John` and the last name `Custom`.



CC BY 4.0

3. Select **Set a password for the user**, enter `Wso2@1234` and finish the wizard.
4. Check the server log. `preparePassword` from your class hashed the password:

```
[2026-11-14 13:41:21,905]  INFO {org.wso2.custom.user.store.CustomUserStoreManager} - Generating hash value using jasypt...
```

CC BY 4.0

5. In the MySQL client, look at the stored password:

```
SELECT UM_USER_NAME, UM_USER_PASSWORD FROM custom_db.UM_USER;
```

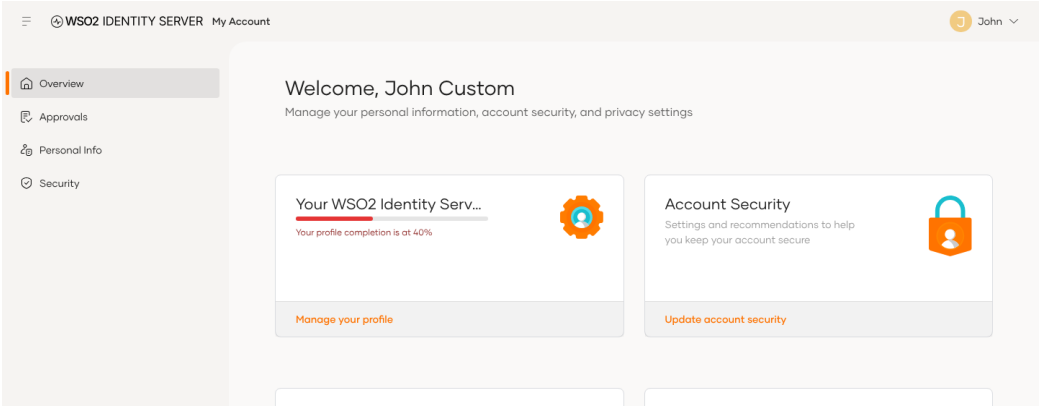
```
mysql> SELECT UM_USER_NAME, UM_USER_PASSWORD FROM custom_db.UM_USER;
+--------------+------------------------------------------------------------------+
| UM_USER_NAME | UM_USER_PASSWORD                                                 |
+--------------+------------------------------------------------------------------+
| john         | Q0ujxQNN6GoqThESvwx5aSqELgtJ980T70FsTx/oXHflY87uyrju3xHoxvMgiten |
+--------------+------------------------------------------------------------------+
```

CC BY 4.0

The value is a Jasypt hash: the random salt and the SHA-256 digest together, in Base64 (64 characters). The value is different every time, even for the same password. The standard JDBC user store would store only the digest and keep the salt in the separate `UM_SALT_VALUE` column.

B. Log in: the password is checked with Jasypt

1. In a private browser window, open My Account (`https://localhost:9443/myaccount`) and log in as `john` with a **wrong** password. The login fails.
2. Log in again as `john` / `Wso2@1234`. My Account opens with **Welcome, John Custom**.



CC BY 4.0

3. Check the server log. `doAuthenticateWithUserName` from your class handled both attempts:

```
[2026-11-14 13:41:54,012]  INFO {org.wso2.custom.user.store.CustomUserStoreManager} - john is authenticated? false
[2026-11-14 13:42:07,748]  INFO {org.wso2.custom.user.store.CustomUserStoreManager} - john is authenticated? true
```

CC BY 4.0

C. After a restart

1. Restart WSO2 Identity Server. The log shows **bundle activated successfully** and **CustomUserStoreManager initialized** again, and `john` can still log in.

Troubleshooting

Problem	Cause and fix
mvn clean install fails with Could not resolve dependencies / Non-resolvable parent POM	The machine cannot reach https://maven.wso2.org or Maven Central. Check the internet connection or the proxy settings in <MAVEN_HOME>/conf/settings.xml.
No "bundle activated successfully" line at startup	The JAR is not in <IS_HOME>/repository/components/dropins, or a different JAR was copied (use the one in target).
CustomUserStoreManager is not listed when you add a user store	The [user_store_mgt] custom_user_stores line is missing or has a typo in the class name. Fix it and restart.
Error while adding the user store, or Communications link failure in the log	MySQL is not running, the URL, username or password is wrong, or the Connector/J JAR is not in <IS_HOME>/repository/components/lib.
Table 'custom_db.UM_USER' doesn't exist	dbscripts/mysql.sql was not run against custom_db (Step 4).
CUSTOM is not offered when you add a user	Refresh the Users page: a new user store can take a few seconds to load.
Login fails for a user that was created before the custom store was used	Passwords hashed by another user store manager cannot be checked by Jasypt. Create the user again in the CUSTOM user store.

CC BY 4.0